Lab 8: Mobile Robot Kinematics

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1 Overview

In this lab you will simulate the two canonical kinematic models of wheeled robots from the Week 9 lecture — the **unicycle / differential drive** and the **bicycle (car-like)** model — and use them to drive a virtual robot along programmed trajectories.

You will implement the kinematics yourself, integrate the motion numerically, and visualise the resulting paths in matplotlib. By the end of the lab you should have a working simulator that accepts wheel (or steering) commands and produces a pose trajectory `(x, y, )(t)`.

Duration: 2 hours

Software Required:

- Python environment from Lab 0
- Libraries: `numpy`, `matplotlib` (both already installed; Colab link will `pip install` if needed)

2 Learning Objectives

By the end of this lab, you will be able to:

1. Implement the unicycle kinematic equations in Python
2. Integrate the ODE `q̇ = G(q) u` using simple Euler steps to produce a trajectory
3. Convert between differential-drive wheel speeds and unicycle commands `(v, )`
4. Implement the bicycle model with steering-angle input
5. Visualise and compare trajectories for the two models under identical commands

3 Pre-Lab: Model Review

Complete this section **before** coming to lab. You can check your answers at the top of the notebook.

3.1 The Unicycle Model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega$$

Controls: `u = (v, )` — forward speed (m/s) and yaw rate (rad/s).

3.2 Differential-Drive Mapping

Given wheel radius `r` and track width `L`:

$$v = \frac{r}{2}(\dot{\phi}_R + \dot{\phi}_L), \qquad \omega = \frac{r}{L}(\dot{\phi}_R - \dot{\phi}_L)$$

Inverse (from desired `(v,  )` to wheel commands):

$$\dot{\phi}_R = \frac{v + \omega L/2}{r}, \qquad \dot{\phi}_L = \frac{v - \omega L/2}{r}$$

3.3 The Bicycle (Car-Like) Model

Given wheelbase `L`, rear-wheel speed `v`, and steering angle :

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \frac{v\tan\delta}{L}$$

Minimum turning radius: `R_min = L / tan( _max)`.

3.4 Pre-Lab Questions

Answer before lab (will be collected in the notebook):

1. A differential-drive robot has `r = 0.05` m, `L = 0.25` m. The right wheel spins at 10 rad/s, the left at 4 rad/s. Compute `v`, , and the turning radius.
2. A car has wheelbase `L = 2.5` m and is driving at `v = 6 m/s` on a circle of radius 30 m. What steering angle does it need?
3. Starting at `q(0) = (0, 0, 0)`, a unicycle drives for 2 s with `v = 1` m/s, ` = 0.5` rad/s. Roughly sketch the path on paper.

---

4 Part 1: Implement the Unicycle Integrator (30 min)

4.1 Step 1.1 — Write the kinematics function

In the notebook, implement:

```
def unicycle_step(q, u, dt):
    """Advance the unicycle by one Euler step.
    q = (x, y, theta), u = (v, omega), dt = step size.
    Returns the new q."""
    # TODO: compute q_dot from (v, omega) and theta,
    #       then return q + q_dot * dt
```

4.2 Step 1.2 — Integrate a trajectory

Write a loop that starts from `q0 = (0, 0, 0)` and steps the unicycle for `T = 4 s` at `dt = 0.01 s` under the constant commands:

- `(v,  ) = (1.0 m/s, 0.0 rad/s)` → should produce a straight line along +x
- `(v,  ) = (0.0, 0.5)` → should produce pure in-place rotation
- `(v,  ) = (1.0, 0.5)` → should produce a circular arc of radius `R = v/  = 2 m`

Plot all three trajectories on the same (x, y) axes with `plt.axis('equal')`.

---

5 Part 2: Differential-Drive Robot (30 min)

5.1 Step 2.1 — Wheel-to-body mapping

Given `r = 0.04 m`, `L = 0.20 m`, implement:

```
def diff_drive_to_unicycle(phi_dot_R, phi_dot_L, r, L):
    """Return (v, omega) from wheel speeds."""

def unicycle_to_diff_drive(v, omega, r, L):
    """Return (phi_dot_R, phi_dot_L) from body commands."""
```

Check that calling one then the other is the identity.

5.2 Step 2.2 — Drive a rectangle

Write a function that drives the differential-drive robot around a `2 m × 1 m` rectangle starting at `(0, 0, 0)`. Use piecewise-constant wheel commands:

1. Drive forward 2 m
2. Spin in place by +90°
3. Drive forward 1 m
4. Spin in place by +90°
5. Drive forward 2 m
6. Spin in place by +90°
7. Drive forward 1 m

Plot the trajectory. Add a marker every 0.2 s to visualise speed.

5.3 Step 2.3 — Discussion question

In your notebook, answer: *Why can the differential-drive robot execute step 2 (spin in place) but a car-like robot cannot?* Write 2–3 sentences.

---

6 Part 3: Bicycle Model (30 min)

6.1 Step 3.1 — Implement

```
def bicycle_step(q, u, L_wb, dt):
    """Advance the bicycle model by one Euler step.
    q = (x, y, theta), u = (v, delta), L_wb = wheelbase, dt = step size."""
```

6.2 Step 3.2 — Drive a circle

With `L_wb = 2.5 m`, `v = 6 m/s`, `  = 10°`, integrate for 15 s starting from `q0 = (0, 0, 0)`. Confirm that:

- The turning radius matches `R = L / tan( )`
- The lap time matches `T = 2  R / v`

Plot the trajectory and annotate the radius.

6.3 Step 3.3 — Parallel parking preview

Try to drive the bicycle from `(0, 0, 0)` to `(0, 2, 0)` using only   and `v` (no spin in place). You will not succeed in one step — that's the point. Sketch the multi-step maneuver you would need on paper and describe it in your notebook.

---

7 Part 4: Compare the Two Models (30 min)

7.1 Step 4.1 — Same commanded path, different model

Give both the unicycle and the bicycle the same high-level command sequence:

1. Drive forward 5 s at `v = 1` m/s
2. Turn while driving for 3 s with a moderate turn command
3. Drive forward 2 s again

For the unicycle, use `(v,  ) = (1, 0.4)`. For the bicycle (use `L_wb = 1.0 m`), find the steering angle   that produces the same `  = v tan( ) / L_wb` during step 2.

Plot both trajectories overlaid. They should match.

7.2 Step 4.2 — What happens at `v = 0`?

Set `v = 0` for both models, then command a non-zero turn. What happens to each? Explain in 1 sentence.

7.3 Step 4.3 — Minimum turning radius

Find the minimum turning radius of the bicycle model with `L_wb = 1.0 m` and `_max = 45°`. Compare to the unicycle model's minimum turning radius. Which one can reach a given point in fewer maneuvers?

---

8 Wrap-Up

At the end of the lab:

1. Save your notebook with all plots rendered
2. Upload the `.ipynb` to the course page
3. Write 2–3 sentences in the last cell on what surprised you

---

9 Reference

Unicycle: `q̇ = (v cos  , v sin  ,  )`

Differential drive: `v = (r/2)(˙_R + ˙_L),  = (r/L)(˙_R - ˙_L)`

Bicycle: `q̇ = (v cos  , v sin  , v tan( )/L)`

Turning radius: `R = v/` (unicycle) or `R = L/tan` (bicycle)